VALIDATION PLAN
OPERATIONAL QUALIFICATION
COMMUNITY SCRIPT SUITE

JR Validated Environment — Version 1.1.0

Field	Value
Document Number	JR-VP-002
Title	Validation Plan — Operational Qualification, Community Script Suite
System	JR Validated Environment
System Version	1.1.0
Document Version	1.0
Status	Draft
Effective Date	2026-03-15
Author	Joep Rous
Reviewer	[Name]
Approver	[Name]

Version History

Version	Date	Author	Description
1.0	2026-03-15	Joep Rous	Initial release. Covers all 22 R and 2 Python community scripts in v1.1.0.

Approval Signatures

Role	Name	Signature	Date
Author			
Reviewer			
Approver (QA)			

1. Purpose

This document defines the Validation Plan for the Operational Qualification (OQ) of the community script suite included in version 1.1.0 of the JR Validated Environment. It specifies the user requirements, test cases, acceptance criteria, and traceability between requirements and tests needed to demonstrate that each script performs correctly for its intended use.

Execution of the tests defined in this plan, together with satisfactory results and documented evidence, constitutes the OQ for the community script suite.

2. Scope

2.1 In Scope

- All 22 R scripts in the R/ directory (excluding jrhello.R)
- All 2 Python scripts in the Python/ directory (excluding jrhello.py)
- Correct computation of outputs for valid inputs
- Correct rejection of invalid inputs with informative error messages
- Correct file I/O behaviour where applicable (CSV input, CSV/PNG output)
- Verification that all R scripts enforce execution within the jrrun validated environment by checking RENV_PATHS_ROOT at startup

2.2 Out of Scope

- Infrastructure scripts in bin/ and admin/ (covered by IQ; see JR-VP-001)
- Demo scripts jrhello.R and jrhello.py
- Performance qualification (PQ) — end-to-end verification studies using production data are conducted separately under the design verification protocol
- User acceptance testing (UAT)
- Validation of R and Python language interpreters (assumed qualified by their respective maintainers)
- Validation of third-party packages (tolerance, MASS, e1071, ggplot2, survival, nortest, outliers) — package integrity is confirmed at installation via SHA256 and is documented in the IQ evidence report (JR-IQ-001)

3. Regulatory Basis and References

3.1 Regulations

Reference	Title
21 CFR Part 820, §820.70(i)	Quality System Regulation — Automated Data Processing
21 CFR Part 11	Electronic Records; Electronic Signatures

3.2 FDA Guidance

Reference	Title	Date
FDA GPSV	General Principles of Software Validation; Final Guidance for Industry and FDA Staff	January 2002
FDA CSA	Computer Software Assurance for Production and Quality System Software; Draft Guidance	September 2022

3.3 Standards

Reference	Title
ASTM F3172-15	Standard Guide for Design Verification Device Size and Sample Size Selection for Endovascular Devices
ISO 13485:2016	Medical devices — Quality management systems

3.4 Internal Documents

Document Number	Title
JR-VP-001	Validation Plan — Installation Qualification, JR Validated Environment v1.0.0
JR-IQ-001	IQ Execution Evidence (file: docs/IQ_validation_20260311_205146.txt)

4. Definitions and Abbreviations

Term	Definition
IQ	Installation Qualification — verifies that the system is installed correctly and that the environment is complete and consistent
OQ	Operational Qualification — verifies that the system operates correctly within specified limits across the range of anticipated inputs
PQ	Performance Qualification — verifies that the system performs consistently under real conditions of use
UR	User Requirement — a statement of what the system must do from the user's perspective
TC	Test Case — a specific set of inputs, execution conditions, and expected results used to verify a user requirement
jrrun	The validated zsh entry-point script that sets the controlled environment before executing any community script
renv	R package management system; used to isolate the validated R library
venv	Python virtual environment; used to isolate the validated Python library
CSV	Comma-separated values file; two-column format (id, value) as defined by the jrc suite
TOST	Two One-Sided Tests — statistical method for equivalence testing
MLE	Maximum Likelihood Estimation
LoA	Limits of Agreement (Bland-Altman method)
TI	Statistical Tolerance Interval
LSL	Lower Specification Limit
USL	Upper Specification Limit

5. System Description

5.1 Software Description

The JR Validated Environment is a controlled R and Python execution environment for medical device design verification and statistical analysis. It consists of a controlled local R package repository (miniCRAN via Dropbox), a pinned isolated R library managed by renv, a pinned isolated Python virtual environment, the jrrun entry-point script which enforces the validated environment and logs every execution, and a suite of community scripts (jrc_*) for sample size calculation, diagnostic analysis, statistical analysis, design verification, data generation, and data conversion. This document covers the community script suite only. The environment infrastructure was qualified in JR-IQ-001.

5.2 Intended Use

The community scripts are intended for use by engineers and scientists in the design verification phase of medical device development. Intended uses include:

- Calculating minimum sample sizes for discrete and continuous design verification
- Assessing the statistical properties of pilot and verification datasets
- Computing and reporting process capability, outliers, and normality
- Performing method comparison studies (Bland-Altman)
- Performing reliability and lifetime analysis (Weibull)
- Verifying measurement datasets against specification limits using statistical tolerance intervals
- Generating synthetic datasets for validation and testing purposes
- Converting raw instrument data files into the standard jrc CSV format

Scripts produce console output and, where applicable, PNG plots and CSV files. All outputs are intended to support engineering reports and design history files (DHF) for FDA submissions.

5.3 User Profile

Primary users are mechanical, biomedical, or quality engineers with working knowledge of design verification testing but who are not expected to have deep statistical programming expertise. Scripts are invoked from the macOS Terminal via wrapper commands (e.g., jrc_ss_discrete 0.99 0.95).

5.4 Hardware and Software Environment

Component	Specification
Operating system	macOS (arm64 / aarch64-apple-darwin)
R version	4.5 (as specified in admin/r_version.txt)
Python version	3.11.9 (as specified in admin/python_version.txt)
R packages	As listed in admin/R_requirements.txt and admin/renv.lock
Python packages	As listed in admin/python_requirements.txt
Entry point	bin/jrrun

6. Risk Assessment and Validation Level

6.1 Risk Classification

The community scripts are used to compute sample sizes, process statistics, and tolerance intervals that directly inform design verification conclusions reported in FDA submissions. Incorrect results could lead to underpowered verification studies that fail to detect non-conforming processes, incorrect pass/fail verdicts in design verification reports, or erroneous statistical claims in regulatory submissions. This represents a high-consequence failure mode affecting patient safety indirectly via the quality of design verification evidence. Per the FDA General Principles of Software Validation (GPSV 2002), the level of validation effort is commensurate with this risk.

6.2 Validation Strategy

Risk Factor	Assessment	Consequence
Computational errors in sample size scripts	Medium probability	High: incorrect N in DHF
Incorrect file I/O (wrong column, wrong delimiter)	Medium probability	High: analysis performed on wrong data
Silent failure (non-zero exit, no error message)	Low probability	Medium: user unaware of failure
Incorrect pass/fail verdict in jrc_verify_attr	Low probability	High: incorrect verification conclusion

Strategy: Full OQ testing of all scripts, covering both primary (happy-path) execution paths and boundary/error-handling paths. Numerical outputs of pure-computation scripts are verified against independently calculated reference values where feasible.

7. Roles and Responsibilities

Role	Responsibility
Author	Writes and maintains this Validation Plan
Reviewer	Reviews test specifications for completeness, accuracy, and regulatory adequacy
QA Approver	Approves this plan prior to OQ execution
Test Executor	Executes the OQ test suite (admin_oq) and documents results
QA Witness	Reviews execution evidence and signs the OQ execution report

The Test Executor and Author may be the same person. The QA Approver must be independent of the Author for this document.

8. Validation Approach

8.1 Overall Strategy

Validation of the JR Validated Environment follows a three-phase IQ/OQ/PQ model:

Phase	Description	Status
IQ	Verifies correct installation: R version, Python version, all required packages installed at pinned versions, SHA256 integrity of all scripts	Completed — JR-IQ-001
OQ	Verifies correct operation of each community script across the range of anticipated inputs (this document)	Planned
PQ	Verifies end-to-end performance under real conditions; executed under individual design verification protocols	Out of scope here

8.2 OQ Execution

OQ tests are executed by the automated test runner admin_oq, which:

1. Activates the OQ Python virtual environment (~/.venvs/MyProject_oq/)
2. Invokes pytest against the test suite in oq/
3. For each test case, calls the community script as a subprocess via jrrun
4. Captures exit code, stdout, and stderr
5. Evaluates pass/fail against the criteria defined in Section 13
6. Writes a timestamped qualification report to ~/.jrscript/MyProject/validation/

All test cases must pass for the OQ to be considered successful. A single failing test case constitutes an OQ failure and must be resolved before the system is released to users.

8.3 Sequence of Execution

7. Generate and commit test data files to oq/data/ (see Section 11)
8. Obtain QA approval of this plan
9. Execute admin_oq on the target machine

10. Review the qualification report
11. Document any deviations (see Section 15)
12. Obtain QA sign-off on the execution evidence

9. User Requirements

The following user requirements define what the community script suite must do from the user's perspective. Each requirement is uniquely identified and traceable to one or more OQ test cases in the Requirements Traceability Matrix (Section 14).

9.1 Sample Size Requirements

ID	Requirement
UR-001	The system shall compute the minimum sample size for discrete (pass/fail) design verification for a given proportion and confidence level, using the exact binomial (chi-squared) method, for 0 to 10 allowed failures.
UR-002	The system shall compute the achieved proportion for a discrete test result given confidence level, sample size, and number of failures.
UR-003	The system shall compute the minimum sample size for continuous attribute design verification using statistical tolerance intervals, for 1-sided and 2-sided specifications, applying Box-Cox transformation when data are non-normal.
UR-004	The system shall verify whether a user-specified planned sample size meets the tolerance interval requirement for a given dataset and specification.
UR-005	The system shall compute the achieved proportion coverage for a continuous attribute verification dataset using tolerance intervals.
UR-006	The system shall compute the minimum pilot sample size required to obtain a reliable estimate of the process standard deviation, for 1-sided and 2-sided specification cases.
UR-007	The system shall compute the minimum sample size for a paired comparison study for a given detectable difference, standard deviation of differences, and test direction (1- or 2-sided).
UR-008	The system shall compute the minimum sample size for equivalence testing (TOST) for a given equivalence margin, standard deviation, and test direction.
UR-009	The system shall compute the minimum sample size for fatigue and lifetime testing using a Weibull reliability model, for a given reliability, confidence, shape parameter, and acceleration factor, for 0 to 5 allowed failures.
UR-010	The system shall provide Gauge R&R study design guidance (operators x replicates combinations) for a given target %GRR and reference type (process variation or tolerance).

9.2 Diagnostic Requirements

ID	Requirement
UR-011	The system shall test a dataset for normality using skewness, Shapiro-Wilk, and Anderson-Darling methods, and shall attempt a Box-Cox transformation if the data are non-normal, reporting the result and recommended approach.
UR-012	The system shall detect outliers in a dataset using the Grubbs test (iterative) and the IQR method, reporting the row identifiers of flagged observations.
UR-013	The system shall compute process capability indices (Cp, Cpk, Pp, Ppk) with 95% confidence intervals for a dataset and one or two specification limits.
UR-014	The system shall compute a descriptive statistics summary for a dataset column, including: N, mean, median, SD, variance, CV, min, max, range, 5th/25th/75th/95th percentiles, IQR, skewness, excess kurtosis, and 95% CI on the mean.

9.3 Statistical Analysis Requirements

ID	Requirement

UR-015	The system shall perform a Bland-Altman method comparison analysis for two paired datasets, reporting bias, SD of differences, limits of agreement with 95% CIs, and proportional bias test result, and shall save a Bland-Altman plot as a PNG file.
UR-016	The system shall fit a 2-parameter Weibull distribution to lifetime data including right-censored observations using MLE, reporting shape and scale parameters with 95% CIs and B1, B10, and B50 life estimates with 95% CIs, and shall save a Weibull probability plot as a PNG file.

9.4 Verification Requirements

ID	Requirement
UR-017	The system shall compute a statistical tolerance interval for a verification dataset for a given proportion and confidence, compare the tolerance limits against one or two specification limits, report a pass or fail verdict for each limit, apply Box-Cox transformation when data are non-normal, and save a histogram PNG file.

9.5 Data Generation Requirements

ID	Requirement
UR-018	The system shall generate reproducible synthetic datasets from the following distributions: normal, log-normal, chi-squared scaled (sqrt), Weibull (boxcox), and uniform. Generated CSV files shall have two columns (id, value). When a seed is provided, repeated calls with identical parameters shall produce identical output.

9.6 Data Conversion Requirements

ID	Requirement
UR-019	The system shall convert a multi-column delimited file to the standard jrc two-column CSV format (id, value) by extracting a user-specified column (by name or 1-based number) with configurable header line skipping and delimiter selection (tab, space, comma, or auto-detect). Non-numeric values shall be skipped with a warning.
UR-020	The system shall convert a single-column plain text file (one numeric value per line) to the standard jrc two-column CSV format (id, value), with optional line range selection. Non-numeric and empty lines shall be skipped with a warning.

9.7 General Requirements (Applicable to All Scripts)

ID	Requirement
UR-021	The system shall validate all input arguments before computation. On receipt of invalid input (wrong type, out-of-range value, missing argument, file not found, column not found), the system shall exit with a non-zero exit code and print an informative error message that identifies the invalid argument or missing resource.
UR-022	All R scripts shall verify at startup that the RENV_PATHS_ROOT environment variable is set, and shall halt with an informative error if it is not, preventing execution outside the jrrun validated environment.

10. Test Environment

Item	Specification
Test runner	pytest (version per oq/requirements.txt)
Test execution method	Subprocess calls via jrrun; exit codes and stdout/stderr captured
OQ venv	~/.venvs/MyProject_oq/ — separate from user venv
OQ venv requirements	oq/requirements.txt (frozen; not modified after OQ execution)
R environment	As qualified in JR-IQ-001
Python environment	As qualified in JR-IQ-001

Test data location	oq/data/ (committed to the repository)
Evidence output	~/.jrscript/MyProject/validation/oq_<datetime>.txt
Machine	The machine on which IQ was executed, or a machine where IQ has been completed

11. Test Data

The following files shall be committed to oq/data/ before OQ execution. All CSV files use the standard two-column format (id, value) unless otherwise noted.

File	Content	Generation Method
normal_n30_mean10_sd1_seed42.csv	30 values, N(mean=10, sd=1), seed 42	jrc_gen_normal 30 10 1 oq/data 42
skewed_n30_lognormal_seed42.csv	30 values, log-normal (meanlog=2, sdlog=0.5), seed 42	jrc_gen_lognormal 30 2.0 0.5 oq/data 42
outlier_n30_seed42.csv	As normal_n30_mean10_sd1_seed42.csv with row 15 replaced by 15.0 (≥5σ outlier)	Generated then manually edited
bland_altman_method1_seed42.csv	25 values, N(mean=10, sd=1), seed 42	jrc_gen_normal 25 10 1 oq/data 42 (rename)
bland_altman_method2_seed42.csv	Same 25 method1 values plus N(mean=0.3, sd=0.2) noise, seed 99	oq/data/generate_ba_method2.R
weibull_n20_seed42.csv	20 rows; columns id, cycles, status; 15 failures, 5 censored; Weibull(shape=2, scale=1000), seed 42	oq/data/generate_weibull.R
method1_short.csv	Rows 1-10 of bland_altman_method1_seed42.csv	Subset of method1 file
all_censored.csv	10 rows; columns id, cycles, status; all status=0	Prepared manually
neg_times.csv	10 rows; columns id, cycles, status; row 1 cycles = -100	Prepared manually
bad_status.csv	10 rows; columns id, cycles, status; status values include 2	Prepared manually
convert_multicolumn.txt	Tab-delimited; 3 metadata header lines; columns SampleID, ForceN, Temperature; 20 data rows	Prepared manually
convert_singlecolumn.txt	200 lines, one numeric value per line, values from N(5, 0.5)	Prepared manually

Data integrity: SHA256 checksums of all files in oq/data/ shall be recorded in oq/data/checksums.sha256 and verified by admin_oq before test execution begins.

12. OQ Test Specifications

Conventions

Command — the command as typed by the user (via the jrrun wrapper). Full paths to data files are replaced by filename only for readability; admin_oq uses absolute paths.

Expected output — text that must appear on stdout or stderr. R scripts use message() for all output (captured on stderr). Python scripts use print() (stdout).

Pass criterion — the automated assertion made by the test runner.

All test case IDs follow the pattern TC-<SCRIPT>-<NNN>.

12.1 jrc_ss_discrete [Tests UR-001, UR-021]

TC-DISC-001

Tests: UR-001
Command: `jrc_ss_discrete 0.99 0.95`
Rationale: For P=0.99, C=0.95, f=0: N = ceil(log(0.05)/log(0.99)) = 299 (exact binomial formula).
Pass criterion: Exit code 0; '299' present in output on the f=0 row.

TC-DISC-002

Tests: UR-001
Command: `jrc_ss_discrete 0.99 0.80`
Rationale: Lower confidence reduces required N.
Pass criterion: Exit code 0; f=0 N is a positive integer less than 299.

TC-DISC-003

Tests: UR-021
Command: `jrc_ss_discrete 1.5 0.95`
Pass criterion: Exit code non-zero; output contains 'proportion'.

TC-DISC-004

Tests: UR-021
Command: `jrc_ss_discrete 0.99 0`
Pass criterion: Exit code non-zero; output contains 'confidence'.

TC-DISC-005

Tests: UR-021
Command: `jrc_ss_discrete 0.99`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.2 jrc_ss_discrete_ci [Tests UR-002, UR-021]

TC-DISCICI-001

Tests: UR-002
Command: `jrc_ss_discrete_ci 0.95 299 0`
Rationale: At N=299, f=0, C=0.95 the achieved proportion must be ≥ 0.99.
Pass criterion: Exit code 0; reported achieved proportion ≥ 0.99.

TC-DISCICI-002

Tests: UR-002
Command: `jrc_ss_discrete_ci 0.95 299 1`
Rationale: One failure reduces achieved proportion.
Pass criterion: Exit code 0; achieved proportion < that reported in TC-DISCICI-001.

TC-DISCICI-003

Tests: UR-021
Command: `jrc_ss_discrete_ci 0.95 10 15`
Pass criterion: Exit code non-zero; output references 'f' or 'n'.

TC-DISCICI-004

Tests: UR-021
Command: `jrc_ss_discrete_ci 0.95 299`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.3 jrc_ss_attr [Tests UR-003, UR-021]

TC-ATTR-001

Tests: UR-003
Command: `jrc_ss_attr 0.95 0.95 normal_n30_mean10_sd1_seed42.csv value 9.0 -`
Pass criterion: Exit code 0; output contains a positive integer minimum N ≥ 10; output contains '✅'.

TC-ATTR-002

Tests: UR-003
Command: `jrc_ss_attr 0.95 0.95 normal_n30_mean10_sd1_seed42.csv value - 11.0`
Pass criterion: Exit code 0; output contains a positive integer minimum N ≥ 10.

TC-ATTR-003

Tests: UR-003
Command: `jrc_ss_attr 0.95 0.95 normal_n30_mean10_sd1_seed42.csv value 9.0 11.0`
Rationale: 2-sided tolerance interval requires more samples than 1-sided.
Pass criterion: Exit code 0; 2-sided minimum N ≥ 1-sided N from TC-ATTR-001.

TC-ATTR-004

Tests: UR-021
Command: `jrc_ss_attr 0.95 0.95 normal_n30_mean10_sd1_seed42.csv value 11.0 9.0`
Pass criterion: Exit code non-zero; output contains 'spec2'.

TC-ATTR-005

Tests: UR-021
Command: `jrc_ss_attr 0.95 0.95 normal_n30_mean10_sd1_seed42.csv value - -`
Pass criterion: Exit code non-zero.

TC-ATTR-006

Tests: UR-021
Command: `jrc_ss_attr 0.95 0.95 nonexistent.csv value 9.0 -`
Pass criterion: Exit code non-zero; output contains 'not found'.

TC-ATTR-007

Tests: UR-021
Command: `jrc_ss_attr 0.95 0.95 normal_n30_mean10_sd1_seed42.csv badcol 9.0 -`
Pass criterion: Exit code non-zero; output contains 'not found' or 'Available'.

12.4 jrc_ss_attr_check [Tests UR-004, UR-021]

TC-ATTRCK-001

Tests: UR-004
Command: `jrc_ss_attr_check 0.95 0.95 normal_n30_mean10_sd1_seed42.csv value 9.0 - 50`
Pass criterion: Exit code 0; output contains a pass indicator ('✅' or 'PASS').

TC-ATTRCK-002

Tests: UR-004
Command: `jrc_ss_attr_check 0.95 0.95 normal_n30_mean10_sd1_seed42.csv value 9.0 - 5`
Pass criterion: Exit code 0; output contains a fail indicator ('❌' or 'FAIL').

TC-ATTRCK-003

Tests: UR-021

Command: `jrc_ss_attr_check 0.95 0.95 normal_n30_mean10_sd1_seed42.csv value 9.0 -`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.5 jrc_ss_attr_ci [Tests UR-005, UR-021]

TC-ATTRCI-001

Tests: UR-005
Command: `jrc_ss_attr_ci 0.95 normal_n30_mean10_sd1_seed42.csv value 9.0 -`
Pass criterion: Exit code 0; output contains a proportion value between 0 and 1 (exclusive).

TC-ATTRCI-002

Tests: UR-005
Command: `jrc_ss_attr_ci 0.95 normal_n30_mean10_sd1_seed42.csv value 9.0 11.0`
Rationale: 2-sided interval is more restrictive than 1-sided.
Pass criterion: Exit code 0; 2-sided proportion ≤ proportion from TC-ATTRCI-001.

TC-ATTRCI-003

Tests: UR-021
Command: `jrc_ss_attr_ci 0.95 nonexistent.csv value 9.0 -`
Pass criterion: Exit code non-zero; output contains 'not found'.

12.6 jrc_ss_sigma [Tests UR-006, UR-021]

TC-SIGMA-001

Tests: UR-006
Command: `jrc_ss_sigma 1.5 9.0 -`
Pass criterion: Exit code 0; output contains a table of N values for multiple power/confidence combinations.

TC-SIGMA-002

Tests: UR-006
Command: `jrc_ss_sigma 1.5 9.0 11.0`
Rationale: 2-sided hypothesis test requires larger N than 1-sided.
Pass criterion: Exit code 0; N values ≥ corresponding values from TC-SIGMA-001.

TC-SIGMA-003

Tests: UR-021
Command: `jrc_ss_sigma -1.0 9.0 -`
Pass criterion: Exit code non-zero.

TC-SIGMA-004

Tests: UR-021
Command: `jrc_ss_sigma 1.5`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.7 jrc_ss_paired [Tests UR-007, UR-021]

TC-PAIRED-001

Tests: UR-007
Command: `jrc_ss_paired 0.5 1.0 2`
Pass criterion: Exit code 0; output contains a table of N values; all N ≥ 10.

TC-PAIRED-002

Tests: UR-007
Command: `jrc_ss_paired 0.5 1.0 1`

Rationale: 1-sided test requires fewer samples than 2-sided.
Pass criterion: Exit code 0; N values strictly less than corresponding values from TC-PAIRED-001.

TC-PAIRED-003

Tests: UR-021
Command: `jrc_ss_paired 0.5 1.0 3`
Pass criterion: Exit code non-zero.

TC-PAIRED-004

Tests: UR-021
Command: `jrc_ss_paired 0.5 0 2`
Pass criterion: Exit code non-zero.

TC-PAIRED-005

Tests: UR-021
Command: `jrc_ss_paired 0.5`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.8 jrc_ss_equivalence [Tests UR-008, UR-021]

TC-EQUIV-001

Tests: UR-008
Command: `jrc_ss_equivalence 0.5 1.0 2`
Pass criterion: Exit code 0; output contains 'TOST' or 'equivalence' (case-insensitive); all N ≥ 10.

TC-EQUIV-002

Tests: UR-008
Command: `jrc_ss_equivalence 0.5 1.0 1`
Rationale: 1-sided non-inferiority requires fewer samples.
Pass criterion: Exit code 0; N values < corresponding values from TC-EQUIV-001.

TC-EQUIV-003

Tests: UR-021
Command: `jrc_ss_equivalence 0.5 1.0 0`
Pass criterion: Exit code non-zero.

TC-EQUIV-004

Tests: UR-021
Command: `jrc_ss_equivalence 0.5`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.9 jrc_ss_fatigue [Tests UR-009, UR-021]

TC-FAT-001

Tests: UR-009
Command: `jrc_ss_fatigue 0.90 0.95 2.0 1.0`
Pass criterion: Exit code 0; output contains a table for f=0..5; f=0 row has the largest N.

TC-FAT-002

Tests: UR-009
Command: `jrc_ss_fatigue 0.90 0.95 2.0 2.0`
Rationale: Higher acceleration factor reduces required sample size.
Pass criterion: Exit code 0; f=0 N strictly less than f=0 N from TC-FAT-001.

TC-FAT-003

Tests: UR-021
Command: `jrc_ss_fatigue 1.0 0.95 2.0 1.0`
Pass criterion: Exit code non-zero.

TC-FAT-004

Tests: UR-021
Command: `jrc_ss_fatigue 0.90 0.95 2.0 0.5`
Pass criterion: Exit code non-zero.

TC-FAT-005

Tests: UR-021
Command: `jrc_ss_fatigue 0.90 0.95 2.0`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.10 jrc_ss_gauge_rr [Tests UR-010, UR-021]

TC-GRR-001

Tests: UR-010
Command: `jrc_ss_gauge_rr 10 process 1.0`
Pass criterion: Exit code 0; output contains a table of operators x replicates; output contains '%GRR' and 'ndc'.

TC-GRR-002

Tests: UR-010
Command: `jrc_ss_gauge_rr 10 tolerance 5.0`
Pass criterion: Exit code 0; output contains a table.

TC-GRR-003

Tests: UR-021
Command: `jrc_ss_gauge_rr 10 badtype 1.0`
Pass criterion: Exit code non-zero.

TC-GRR-004

Tests: UR-021
Command: `jrc_ss_gauge_rr 10`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.11 jrc_normality [Tests UR-011, UR-021]

TC-NORM-001

Tests: UR-011
Command: `jrc_normality normal_n30_mean10_sd1_seed42.csv value`
Rationale: Data is N(10,1); normality should be confirmed.
Pass criterion: Exit code 0; output contains 'normal' (case-insensitive) and '✅'.

TC-NORM-002

Tests: UR-011
Command: `jrc_normality skewed_n30_lognormal_seed42.csv value`
Rationale: Log-normal data is non-normal; Box-Cox path should be triggered.
Pass criterion: Exit code 0; output contains 'Box-Cox' or 'not normal'.

TC-NORM-003

Tests: UR-021
Command: `jrc_normality nonexistent.csv value`
Pass criterion: Exit code non-zero; output contains 'not found'.

TC-NORM-004

Tests: UR-021
Command: `jrc_normality normal_n30_mean10_sd1_seed42.csv badcol`
Pass criterion: Exit code non-zero; output contains 'not found' or 'Available'.

TC-NORM-005

Tests: UR-021
Command: `jrc_normality normal_n30_mean10_sd1_seed42.csv`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.12 jrc_outliers [Tests UR-012, UR-021]

TC-OUT-001

Tests: UR-012
Command: `jrc_outliers normal_n30_mean10_sd1_seed42.csv value`
Rationale: Data has no injected outliers; Grubbs and IQR should find none.
Pass criterion: Exit code 0; output contains 'no outlier' (case-insensitive) or a count of 0 flagged observations.

TC-OUT-002

Tests: UR-012
Command: `jrc_outliers outlier_n30_seed42.csv value`
Rationale: Row 15 = 15.0 is ≥ 5σ above the mean and must be detected.
Pass criterion: Exit code 0; output references row '15' or reports ≥ 1 flagged observation.

TC-OUT-003

Tests: UR-021
Command: `jrc_outliers nonexistent.csv value`
Pass criterion: Exit code non-zero.

TC-OUT-004

Tests: UR-021
Command: `jrc_outliers normal_n30_mean10_sd1_seed42.csv`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.13 jrc_capability [Tests UR-013, UR-021]

TC-CAP-001

Tests: UR-013
Command: `jrc_capability normal_n30_mean10_sd1_seed42.csv value 7.0 13.0`
Rationale: Data is N(10,1); spec is ±3σ. Cp ≈ 1.0; Cpk should be positive.
Pass criterion: Exit code 0; output contains 'Cp', 'Cpk', 'Pp', 'Ppk'; reported Cpk > 0.

TC-CAP-002

Tests: UR-013
Command: `jrc_capability normal_n30_mean10_sd1_seed42.csv value - 13.0`
Pass criterion: Exit code 0; output contains 'Cpk' or 'Ppk'; no computation error.

TC-CAP-003

Tests: UR-021
Command: `jrc_capability normal_n30_mean10_sd1_seed42.csv value - -`
Pass criterion: Exit code non-zero.

TC-CAP-004

Tests: UR-021

Command: `jrc_capability nonexistent.csv value 7.0 13.0`
Pass criterion: Exit code non-zero.

12.14 jrc_descriptive [Tests UR-014, UR-021]

TC-DESC-001

Tests: UR-014
Command: `jrc_descriptive normal_n30_mean10_sd1_seed42.csv value`
Pass criterion: Exit code 0; output contains 'mean', 'median', 'SD', 'min', 'max', 'skewness'; reported mean is within ±0.5 of 10.0.

TC-DESC-002

Tests: UR-021
Command: `jrc_descriptive nonexistent.csv value`
Pass criterion: Exit code non-zero; output contains 'not found'.

TC-DESC-003

Tests: UR-021
Command: `jrc_descriptive normal_n30_mean10_sd1_seed42.csv badcol`
Pass criterion: Exit code non-zero.

TC-DESC-004

Tests: UR-021
Command: `jrc_descriptive normal_n30_mean10_sd1_seed42.csv`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.15 jrc_bland_altman [Tests UR-015, UR-021]

TC-BA-001

Tests: UR-015
Command: `jrc_bland_altman bland_altman_method1_seed42.csv value bland_altman_method2_seed42.csv value`
Rationale: Method 2 was generated with a mean bias of +0.3; reported bias should be approximately +0.3.
Pass criterion: Exit code 0; output contains 'Bias'; output contains 'Limits of Agreement' or 'LoA'; reported bias is within ±0.2 of 0.3; PNG file created in directory of method1 file.

TC-BA-002

Tests: UR-015
Command: `jrc_bland_altman bland_altman_method1_seed42.csv value bland_altman_method2_seed42.csv value`
Pass criterion: Exit code 0; output contains 'proportional bias' (case-insensitive).

TC-BA-003

Tests: UR-021
Command: `jrc_bland_altman method1_short.csv value bland_altman_method2_seed42.csv value`
Rationale: method1_short.csv has 10 rows; method2 has 25 rows — mismatch must be detected.
Pass criterion: Exit code non-zero; output contains 'different numbers' or 'observations'.

TC-BA-004

Tests: UR-021
Command: `jrc_bland_altman nonexistent.csv value bland_altman_method2_seed42.csv value`
Pass criterion: Exit code non-zero.

TC-BA-005

Tests: UR-021
Command: `jrc_bland_altman bland_altman_method1_seed42.csv value bland_altman_method2_seed42.csv`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.16 jrc_weibull [Tests UR-016, UR-021]

TC-WEIB-001

Tests: UR-016
Command: `jrc_weibull weibull_n20_seed42.csv cycles status`
Rationale: Data generated from Weibull(shape=2, scale=1000); MLE estimates should be within ±50% of true values given N=20.
Pass criterion: Exit code 0; output contains 'beta' and 'eta'; reported beta is in range (1.0, 3.0); output contains 'B1', 'B10', 'B50'; PNG file created.

TC-WEIB-002

Tests: UR-021
Command: `jrc_weibull all_censored.csv cycles status`
Rationale: All units censored — fewer than 2 failures.
Pass criterion: Exit code non-zero; output contains 'failure' or 'At least 2'.

TC-WEIB-003

Tests: UR-021
Command: `jrc_weibull neg_times.csv cycles status`
Rationale: Row 1 has cycles = -100.
Pass criterion: Exit code non-zero; output contains 'positive'.

TC-WEIB-004

Tests: UR-021
Command: `jrc_weibull bad_status.csv cycles status`
Rationale: Status column contains value 2.
Pass criterion: Exit code non-zero; output contains '0' and '1'.

TC-WEIB-005

Tests: UR-021
Command: `jrc_weibull nonexistent.csv cycles status`
Pass criterion: Exit code non-zero.

TC-WEIB-006

Tests: UR-021
Command: `jrc_weibull weibull_n20_seed42.csv cycles`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.17 jrc_verify_attr [Tests UR-017, UR-021]

TC-VER-001

Tests: UR-017
Command: `jrc_verify_attr 0.95 0.95 normal_n30_mean10_sd1_seed42.csv value 7.0 -`
Rationale: Data is N(10,1); LSL=7.0 is 3σ below mean; TI lower limit should exceed 7.0.
Pass criterion: Exit code 0; output contains '✅' and 'Lower Tolerance Limit greater than Lower Spec Limit'.

TC-VER-002

Tests: UR-017

Command: `jrc_verify_attr 0.95 0.95 normal_n30_mean10_sd1_seed42.csv value 9.8 -`
Rationale: LSL=9.8 is close to the mean; TI lower limit should be below 9.8.
Pass criterion: Exit code 0; output contains '❌' and 'Lower Tolerance Limit less than Lower Spec Limit'.

TC-VER-003

Tests: UR-017
Command: `jrc_verify_attr 0.95 0.95 normal_n30_mean10_sd1_seed42.csv value 7.0 13.0`
Pass criterion: Exit code 0; output contains '✅' and 'inside Spec'.

TC-VER-004

Tests: UR-017
Command: `jrc_verify_attr 0.95 0.95 skewed_n30_lognormal_seed42.csv value 1.0 -`
Rationale: Log-normal data triggers Box-Cox path; tolerance limit must be in original units.
Pass criterion: Exit code 0; output contains 'boxcox' or 'Box-Cox'; no computation error.

TC-VER-005

Tests: UR-021
Command: `jrc_verify_attr 0.95 0.95 normal_n30_mean10_sd1_seed42.csv value 11.0 9.0`
Pass criterion: Exit code non-zero; output contains 'spec2'.

TC-VER-006

Tests: UR-017
Command: `jrc_verify_attr 0.95 0.95 normal_n30_mean10_sd1_seed42.csv value 7.0 -`
Rationale: PNG output is required per UR-017.
Pass criterion: A file matching '*_tolerance.png' is present in the directory of the input CSV after execution.

TC-VER-007

Tests: UR-021
Command: `jrc_verify_attr 0.95 0.95 nonexistent.csv value 7.0 -`
Pass criterion: Exit code non-zero.

TC-VER-008

Tests: UR-021
Command: `jrc_verify_attr 0.95 0.95 normal_n30_mean10_sd1_seed42.csv value 7.0`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.18 jrc_gen_normal [Tests UR-018, UR-021]

TC-GEN-NO-001

Tests: UR-018
Command: `jrc_gen_normal 50 10.0 1.0 <output_dir> 42`
Rationale: With seed 42 the output is deterministic.
Pass criterion: Exit code 0; CSV created; header is 'id,value'; exactly 50 data rows; all values in 'value' column present; no constraint on range.

TC-GEN-NO-002

Tests: UR-018
Command: `Run TC-GEN-NO-001 twice; compare SHA256 of both outputs.`
Rationale: Reproducibility requirement per UR-018.
Pass criterion: SHA256 checksums of both output files are identical.

TC-GEN-NO-003

Tests: UR-021

Command: `jrc_gen_normal 50 10.0 0 <output_dir> 42`
Rationale: Invalid argument: sd <= 0.
Pass criterion: Exit code non-zero.

TC-GEN-NO-004

Tests: UR-021
Command: `jrc_gen_normal 50`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.19 jrc_gen_lognormal [Tests UR-018, UR-021]

TC-GEN-LO-001

Tests: UR-018
Command: `jrc_gen_lognormal 50 2.0 0.5 <output_dir> 42`
Rationale: With seed 42 the output is deterministic.
Pass criterion: Exit code 0; CSV created; header is 'id,value'; exactly 50 data rows; all values in 'value' column are strictly positive.

TC-GEN-LO-002

Tests: UR-018
Command: `Run TC-GEN-LO-001 twice; compare SHA256 of both outputs.`
Rationale: Reproducibility requirement per UR-018.
Pass criterion: SHA256 checksums of both output files are identical.

TC-GEN-LO-003

Tests: UR-021
Command: `jrc_gen_lognormal 50 2.0 0 <output_dir> 42`
Rationale: Invalid argument: sdlog <= 0.
Pass criterion: Exit code non-zero.

TC-GEN-LO-004

Tests: UR-021
Command: `jrc_gen_lognormal 50`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.20 jrc_gen_sqrt [Tests UR-018, UR-021]

TC-GEN-SQ-001

Tests: UR-018
Command: `jrc_gen_sqrt 50 3 1.0 <output_dir> 42`
Rationale: With seed 42 the output is deterministic.
Pass criterion: Exit code 0; CSV created; header is 'id,value'; exactly 50 data rows; all values in 'value' column are non-negative.

TC-GEN-SQ-002

Tests: UR-018
Command: `Run TC-GEN-SQ-001 twice; compare SHA256 of both outputs.`
Rationale: Reproducibility requirement per UR-018.
Pass criterion: SHA256 checksums of both output files are identical.

TC-GEN-SQ-003

Tests: UR-021
Command: `jrc_gen_sqrt 50 0 1.0 <output_dir> 42`
Rationale: Invalid argument: df <= 0.

Pass criterion: Exit code non-zero.

TC-GEN-SQ-004

Tests: UR-021
Command: `jrc_gen_sqrt 50`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.21 jrc_gen_boxcox [Tests UR-018, UR-021]

TC-GEN-BO-001

Tests: UR-018
Command: `jrc_gen_boxcox 50 2.0 1000.0 <output_dir> 42`
Rationale: With seed 42 the output is deterministic.
Pass criterion: Exit code 0; CSV created; header is 'id,value'; exactly 50 data rows; all values in 'value' column are strictly positive.

TC-GEN-BO-002

Tests: UR-018
Command: `Run TC-GEN-BO-001 twice; compare SHA256 of both outputs.`
Rationale: Reproducibility requirement per UR-018.
Pass criterion: SHA256 checksums of both output files are identical.

TC-GEN-BO-003

Tests: UR-021
Command: `jrc_gen_boxcox 50 0 1000.0 <output_dir> 42`
Rationale: Invalid argument: shape <= 0.
Pass criterion: Exit code non-zero.

TC-GEN-BO-004

Tests: UR-021
Command: `jrc_gen_boxcox 50`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.22 jrc_gen_uniform [Tests UR-018, UR-021]

TC-GEN-UN-001

Tests: UR-018
Command: `jrc_gen_uniform 50 0.0 10.0 <output_dir> 42`
Rationale: With seed 42 the output is deterministic.
Pass criterion: Exit code 0; CSV created; header is 'id,value'; exactly 50 data rows; all values satisfy 0.0 <= value <= 10.0.

TC-GEN-UN-002

Tests: UR-018
Command: `Run TC-GEN-UN-001 twice; compare SHA256 of both outputs.`
Rationale: Reproducibility requirement per UR-018.
Pass criterion: SHA256 checksums of both output files are identical.

TC-GEN-UN-003

Tests: UR-021
Command: `jrc_gen_uniform 50 10.0 5.0 <output_dir> 42`
Rationale: Invalid argument: max <= min.
Pass criterion: Exit code non-zero.

TC-GEN-UN-004

Tests: UR-021
Command: `jrc_gen_uniform 50`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.23 jrc_convert_csv [Tests UR-019, UR-021]

TC-CCSV-001

Tests: UR-019
Command: `jrc_convert_csv convert_multicolumn.txt ForceN 3`
Rationale: Column by name; auto-detect delimiter; skip 3 header lines.
Pass criterion: Exit code 0; output CSV created; header is 'id,value'; row count equals data rows in source; output contains '✅'.

TC-CCSV-002

Tests: UR-019
Command: `jrc_convert_csv convert_multicolumn.txt 2 3`
Rationale: Column 2 (1-based) = ForceN; result must match TC-CCSV-001.
Pass criterion: Exit code 0; output CSV values identical to TC-CCSV-001.

TC-CCSV-003

Tests: UR-019
Command: `jrc_convert_csv convert_multicolumn.txt ForceN 3 tab`
Rationale: Explicit tab delimiter; result must match TC-CCSV-001.
Pass criterion: Exit code 0; result identical to TC-CCSV-001.

TC-CCSV-004

Tests: UR-021
Command: `jrc_convert_csv convert_multicolumn.txt ForceN 999`
Pass criterion: Exit code non-zero; output contains 'skip_lines'.

TC-CCSV-005

Tests: UR-021
Command: `jrc_convert_csv convert_multicolumn.txt NonExistent 3`
Pass criterion: Exit code non-zero; output contains 'not found'.

TC-CCSV-006

Tests: UR-021
Command: `jrc_convert_csv nonexistent.txt ForceN 0`
Pass criterion: Exit code non-zero; output contains 'not found'.

TC-CCSV-007

Tests: UR-021
Command: `jrc_convert_csv convert_multicolumn.txt ForceN 3 pipe`
Pass criterion: Exit code non-zero; output contains 'delimiter'.

TC-CCSV-008

Tests: UR-021
Command: `jrc_convert_csv convert_multicolumn.txt ForceN`
Pass criterion: Exit code non-zero; output contains 'Usage'.

12.24 jrc_convert_txt [Tests UR-020, UR-021]

TC-CTXT-001

Tests: UR-020
Command: `jrc_convert_txt convert_singlecolumn.txt`
Pass criterion: Exit code 0; output CSV created; header is 'id,value'; exactly 200 data rows; output contains '✅'.

TC-CTXT-002

Tests: UR-020
Command: `jrc_convert_txt convert_singlecolumn.txt 50 100`
Pass criterion: Exit code 0; output CSV has exactly 51 data rows; filename contains 'lines50to100'.

TC-CTXT-003

Tests: UR-020
Command: `jrc_convert_txt convert_singlecolumn.txt 150`
Rationale: No end_line; defaults to end of file (line 200).
Pass criterion: Exit code 0; output CSV has 51 data rows; filename contains 'lines150to200'.

TC-CTXT-004

Tests: UR-021
Command: `jrc_convert_txt convert_singlecolumn.txt 500`
Pass criterion: Exit code non-zero; output contains 'start_line' or 'exceeds'.

TC-CTXT-005

Tests: UR-021
Command: `jrc_convert_txt convert_singlecolumn.txt 100 50`
Pass criterion: Exit code non-zero; output contains 'end_line'.

TC-CTXT-006

Tests: UR-021
Command: `jrc_convert_txt nonexistent.txt`
Pass criterion: Exit code non-zero; output contains 'not found'.

TC-CTXT-007

Tests: UR-021
Command: `jrc_convert_txt`
Pass criterion: Exit code non-zero; output contains 'Usage'.

13. Acceptance Criteria

13.1 Individual Test Case Criterion

A test case passes when all of the following are true:

13. The subprocess exit code matches the expected value (0 for success cases, non-zero for error cases).
14. All required strings are present in the combined stdout + stderr output.
15. Where a file output is asserted, the file exists at the expected path.
16. Where a numerical bound is asserted (e.g., N ≥ 10, reported mean within ±0.5 of 10.0), the extracted value satisfies the bound.

13.2 OQ Acceptance Criterion

The OQ is passed when all 122 test cases pass with no deviations. A single failing test case constitutes an OQ failure and must be resolved before the system is released for use.

13.3 Exclusions

No test cases may be excluded without documented justification reviewed and approved by QA. An exclusion is not the same as a deviation — exclusions must be agreed before OQ execution; deviations are handled after (Section 15).

14. Requirements Traceability Matrix

User Requirement	Test Cases
UR-001 — Discrete sample size	TC-DISC-001, TC-DISC-002, TC-DISC-003, TC-DISC-004, TC-DISC-005
UR-002 — Discrete achieved proportion	TC-DISCICI-001, TC-DISCICI-002, TC-DISCICI-003, TC-DISCICI-004
UR-003 — Attribute sample size	TC-ATTR-001, TC-ATTR-002, TC-ATTR-003, TC-ATTR-004, TC-ATTR-005, TC-ATTR-006, TC-ATTR-007
UR-004 — Attribute check	TC-ATTRCK-001, TC-ATTRCK-002, TC-ATTRCK-003
UR-005 — Attribute achieved proportion	TC-ATTRCI-001, TC-ATTRCI-002, TC-ATTRCI-003
UR-006 — Sigma pilot size	TC-SIGMA-001, TC-SIGMA-002, TC-SIGMA-003, TC-SIGMA-004
UR-007 — Paired comparison size	TC-PAIRED-001, TC-PAIRED-002, TC-PAIRED-003, TC-PAIRED-004, TC-PAIRED-005
UR-008 — Equivalence size	TC-EQUIV-001, TC-EQUIV-002, TC-EQUIV-003, TC-EQUIV-004
UR-009 — Fatigue sample size	TC-FAT-001, TC-FAT-002, TC-FAT-003, TC-FAT-004, TC-FAT-005
UR-010 — Gauge R&R design	TC-GRR-001, TC-GRR-002, TC-GRR-003, TC-GRR-004
UR-011 — Normality testing	TC-NORM-001, TC-NORM-002, TC-NORM-003, TC-NORM-004, TC-NORM-005
UR-012 — Outlier detection	TC-OUT-001, TC-OUT-002, TC-OUT-003, TC-OUT-004
UR-013 — Process capability	TC-CAP-001, TC-CAP-002, TC-CAP-003, TC-CAP-004
UR-014 — Descriptive statistics	TC-DESC-001, TC-DESC-002, TC-DESC-003, TC-DESC-004
UR-015 — Bland-Altman analysis	TC-BA-001, TC-BA-002, TC-BA-003, TC-BA-004, TC-BA-005
UR-016 — Weibull analysis	TC-WEIB-001, TC-WEIB-002, TC-WEIB-003, TC-WEIB-004, TC-WEIB-005, TC-WEIB-006
UR-017 — Tolerance interval verification	TC-VER-001, TC-VER-002, TC-VER-003, TC-VER-004, TC-VER-005, TC-VER-006, TC-VER-007, TC-VER-008
UR-018 — Synthetic data generation	TC-GEN-NO-001..004, TC-GEN-LO-001..004, TC-GEN-SQ-001..004, TC-GEN-BO-001..004, TC-GEN-UN-001..004
UR-019 — CSV conversion	TC-CCSV-001, TC-CCSV-002, TC-CCSV-003, TC-CCSV-004, TC-CCSV-005, TC-CCSV-006, TC-CCSV-007, TC-CCSV-008
UR-020 — Text file conversion	TC-CTXT-001, TC-CTXT-002, TC-CTXT-003, TC-CTXT-004, TC-CTXT-005, TC-CTXT-006, TC-CTXT-007
UR-021 — Input validation and error handling	At least one error-path TC per script (throughout Section 12)
UR-022 — Environment enforcement	TC-GEN-NO-001 (RENV_PATHS_ROOT verified at startup via jrrun)

15. Deviation Handling

A deviation is any departure from the test procedure or any test case result that does not meet the pass criterion defined in Section 13.

15.1 During Execution

If a test case fails during OQ execution:

17. The test executor records the deviation: test case ID, observed result, expected result, and the full output captured by admin_oq.
18. Execution of the remaining test cases continues unless the failure prevents subsequent tests from running.
19. The test executor does not modify test data, test scripts, or community scripts during execution without prior QA approval.

15.2 Deviation Resolution

Each deviation must be resolved before the OQ is closed. Resolution options are:

Resolution	Condition
Correction + retest	A defect in the community script is identified and corrected; affected test cases are re-executed after the correction is committed and the integrity hash is regenerated.
Test case correction	The test case itself contains an error (e.g., wrong expected value); the correction is documented, reviewed, and approved before retesting.
Exclusion with justification	In exceptional circumstances, QA may accept a documented justification for excluding a test case; this requires written approval and must be recorded in the OQ Execution Report.

All deviations and their resolutions shall be documented in the OQ Execution Report.

16. Evidence and Documentation

Successful completion of the OQ produces the following evidence:

Evidence Item	Location	Retained By
This Validation Plan (approved, signed)	docs/oq_validation_plan.docx + signed PDF	QA
Test data files with SHA256 checksums	oq/data/ (committed to repository)	Repository
OQ test suite source code	oq/ (committed to repository)	Repository
OQ execution report (automated, timestamped)	~/.jrscript/MyProject/validation/oq_<datetime>.txt	QA
OQ execution report (copy)	docs/	Repository
Deviation log (if any)	Appended to OQ execution report	QA

All evidence shall be retained for the lifetime of the device plus the period required by applicable regulations (minimum 2 years post-market per 21 CFR 820.180, or per the applicable device-specific regulation).

17. Related Documents

Document	Location
JR-VP-001 — IQ Validation Plan	docs/validation_plan.docx
IQ Execution Evidence (JR-IQ-001)	docs/IQ_validation_20260311_205146.txt
Validation Report template	docs/templates/jr_validation_report_template.docx
CHANGELOG	CHANGELOG.md
Infrastructure Backlog	INFRA_BACKLOG.md
CONTRIBUTING guide	CONTRIBUTING.md

End of Document — JR-VP-002 v1.0